



OPEN Performance and energy optimization of ternary optical computers based on tandem queuing system

Heqiang Zhang¹, Meng Liu¹, Weiwen Liu¹, Wenqiang Shi¹, Shuang Li², Jie Zhang³ & Xianchao Wang^{1,4}✉

As an emerging computer technology with numerous bits, bit-wise allocation, and extensive parallelism, the ternary optical computer (TOC) will play an important role in platforms such as cloud computing and big data. Previous studies on TOC in handling computational request tasks have mainly focused on performance enhancement while ignoring the impact of performance enhancement on power consumption. The main objective of this study is to investigate the optimization trade-off between performance and energy consumption in TOC systems. To this end, the service model of the TOC is constructed by introducing the M/M/1 and M/M/c models in queuing theory, combined with the framework of the tandem queueing system, and the optimization problem is studied by adjusting the processor partitioning strategy and the number of small TOC (STOC) in the service process. The results show that the value of increasing active STOCs is prominent when system performance significantly depends on response time. However, marginal gains decrease as the number of STOCs grows, accompanied by rising energy costs. Based on these findings, this paper constructs a bi-objective optimization model using response time and energy consumption. It proposes an optimization strategy to achieve bi-objective optimization of performance and energy consumption for TOC by identifying the optimal partitioning strategy and the number of active small optical processors for different load conditions.

Keywords Ternary optical computer, Tandem queuing system, Response time, Energy consumption, Bi-objective optimization

Since the initiated exploration in 2003, the research on TOC^{1,2} has made remarkable progress after years of unremitting efforts. For example, the theory of the Decrease-Radix Design Principle³ proposed in 2005 simplified the design of ternary logic operators and laid the theoretical foundation for the development of TOC. Under the guidance of this theory, optical computing devices such as SD11⁴ and SD16⁵ (an optical computer developed by Shanghai University in 2016, with “SD” representing the abbreviation of its Chinese name Shangda) have been designed and manufactured. In addition, TOC has successfully implemented the operation of typical algorithms such as cellular automata⁶, the jacobi iterative algorithm⁷, genetic algorithm⁸, and the particle swarm algorithm⁹. Although certain results have been achieved in improving computational speed and optimizing task processing efficiency, existing research still mainly focuses on performance improvement, while systematic research on the trade-offs between performance and energy consumption is scarce. This is particularly critical for TOC, where energy consumption cannot be overlooked. The relevant theoretical framework has not yet formed a unified system, and the discussion of energy consumption in the existing literature is mostly simplistic or secondary, while the trade-off relationship between performance and energy consumption lacks in-depth analysis, which needs to be further explored and improved.

However, to solve these problems and achieve the bi-objective optimization of performance and energy consumption, the existing research paradigm alone is insufficient. At this point, it is especially important

¹School of Computer and Information Engineering, Fuyang Normal University, Fuyang 236037, Anhui, China.

²College of Information, Mechanical and Electrical Engineering, Shanghai Normal University, Shanghai 200234, China. ³School of Mathematics and Statistics, Fuyang Normal University, Fuyang 236037, Anhui, China. ⁴Anhui Engineering Research Center for Intelligent Computing and Information Innovation, Fuyang 236037, Anhui, China.

✉email: wxcdx@126.com

to provide new perspectives and tools for TOC research with the help of theoretical approaches from other disciplines. Queuing theory, as a mature and widely used mathematical model, provides strong support to solve the performance optimization problem of complex systems.

Queuing theory, initially developed for optimizing early telephone networks¹⁰, has become a fundamental tool for system performance analysis across various domains, including computing^{11,12}. In traditional electronic computing systems, queuing models such as M/M/c have been extensively applied to optimize resource allocation and improve system efficiency¹³.

As an emerging computer technology, the TOC can realize efficient processing of large-scale data by the parallel processing capability and high-speed transmission characteristics of optical components. This makes TOC have important application potential in areas such as large-scale data processing^{14–16}, artificial intelligence, and cloud computing. Compared to traditional electronic components, the low energy consumption and high transmission efficiency of optical components are the keys to achieving high performance and low energy consumption in TOC. However, in the current context of explosive growth in data volume, even TOCs, known for their low energy consumption, inevitably face the challenge of high energy consumption in the face of massive data processing¹⁷. Therefore, effective control of TOC energy consumption becomes the core issue of system optimization and targeted energy-saving. The measures are urgently needed.

To this end, based on the theory of the queueing system, this paper combines the technical characteristics of TOC, mathematically models its service pattern, and proposes a strategy for synergistic optimization of performance and energy consumption.

Related works

The TOC architecture

The emergence of the TOC originates from humanity's persistent exploration of computational tools. Throughout history from ancient counting rods and abacuses to modern electronic computers the essence of computation has relied on manipulating physical states to represent and process information. With technological advancements, optical computing emerged as a novel paradigm, utilizing light's physical properties (intensity, wavelength, phase, propagation direction, and polarization) for information encoding. However, studies revealed critical limitations: intensity, wavelength, and phase suffer from signal attenuation or complex control requirements, whereas propagation direction and polarization orientation demonstrate superior stability and controllability¹⁸.

As shown in Fig. 1, this is the main structure in a TOC unit consisting of two polarizers sandwiching a rotator, but the polarizers of each primitive are oriented differently or the control signals of the rotator are different. This characteristic led to the definition of three stable states: absence of light and two orthogonally polarized light states¹⁹. Such a tri-state framework not only simplifies physical implementation but also enables efficient parallel operations.

During the foundational research phase of the TOC, researchers recognized that its processor supports a vast set of 19683¹⁸ ternary logic operations. Direct implementation of all operators was impractical due to prohibitive hardware complexity. This challenge motivated the Decrease-Radix Design Principle (DRDP)³, which synthesizes arbitrary x-valued logic operations through a minimal set of primitive units. Specifically, the DRDP demonstrates that only $x * x * (x - 1)$ primitives are required to construct all possible x-valued logic operators. For TOC ($x = 3$), merely 18 primitives can combinatorially generate all 19683 ternary operators³.

MSD adder

In TOC, the modified signed-digit (MSD) number system is employed for data encoding and computation, enabling fully parallel carry-free addition. The MSD number system was first proposed by Avizienis et al. in 1961 and represents a signed-digit ternary binary system²⁰. For any real number A , its MSD representation is given by:

$$A = \sum_i a_i \cdot 2^i \quad (1)$$

where $a_i \in \{u, 0, 1\}$, with u representing -1 . Any number (except zero) can have multiple MSD representations. For example:

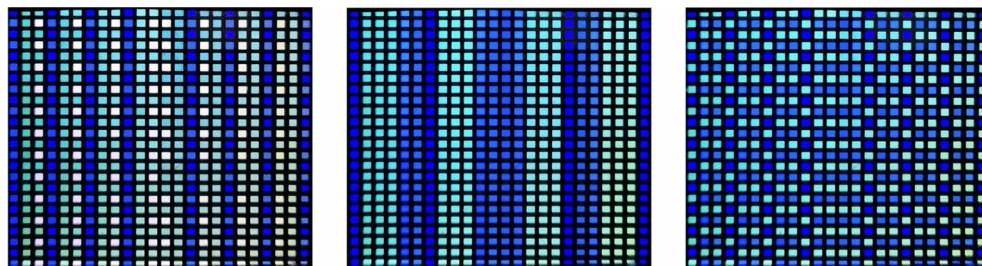


Fig. 1. Main structure of TOC.

T	u	0	1	W	u	0	1	T'	u	0	1	W'	u	0	1
u	u	u	0	u	0	1	0	u	u	0	0	u	0	u	0
0	u	0	1	0	1	0	u	0	0	0	0	0	u	0	1
1	0	1	1	1	0	u	0	1	0	0	1	1	0	1	0

Table 1. Four transformations.

		augend $(14)_{10}$													
		addend $(112)_{10}$													
				ϕ	ϕ	1	u	0	u	1	0	u	0	u	0
Step 1	$T(t_i)$			1	0	1	u	0	u	1	0	ϕ			
	$W(w_i)$			ϕ	u	0	u	0	0	1	u	0			
Step 2	$T'(t'_i)$			0	0	0	u	0	0	1	0	0	ϕ		
	$W'(W'_i)$			ϕ	1	u	1	0	0	u	0	u	0		
Step 3	$T(t_i)$			0	1	u	0	0	0	0	0	u	0		

Table 2. Calculation Example: $14 + 112$.

$$(6)_{10} = (110)_2 = (1u10)_{\text{MSD}} = (1uu10)_{\text{MSD}},$$

$$(-6)_{10} = (u1u0)_{\text{MSD}} = (u11u0)_{\text{MSD}}.$$

From Eq. (1), the following characteristics of the MSD number system can be observed:

- The bitwise negation of an MSD number, following the rules $u \rightarrow 1$, $1 \rightarrow u$, and $0 \rightarrow 0$, yields its additive inverse.
- If the most significant digit is positive, the number is positive; otherwise, it is negative. Thus, the MSD representation does not require a separate sign bit.
- The MSD system exhibits redundancy.
- Positive and negative numbers share the same representation, eliminating the need for one's complement or two's complement representations.

Due to its redundancy, MSD addition and subtraction operations do not involve carry propagation or borrow propagation. To enable fully parallel execution of MSD addition in an optical computing system, four types of transformations are utilized, as shown in Table 1. The following example in Table 2 illustrates the MSD addition process in TOC²¹.

Through the following three steps, the sum of 14 and 112 can be calculated in the MSD number form:

1. Perform both T and W transforms simultaneously. The result of the T transform at the i th position is taken as the carry and should be written at the $(i + 1)$ th position, with the 0th position padded with ϕ .
2. Apply both T' and W' transforms simultaneously to the result of Step 2. The result of the T' transform at the i th position is taken as the carry and should be written at the $(i + 1)$ th position, with the 0th position padded with ϕ .
3. Perform the T transform on the result of Step 2 to obtain the sum of the two MSD numbers.

Converting the third step's result using Eq. (1) yields the decimal number 126.

Performance study on TOC

Previous studies on TOC have primarily focused on performance optimization, aiming to enhance processing speed, reduce response time, and improve computational efficiency. For instance, in order to improve QoS, Zhang Sulan et al. combined the M/M/1 queuing system and used the average response time to analyze and evaluate the performance of TOC²². A simulation model was employed to demonstrate the impact of various indicators on response time. The results indicate that computational and network transmission speeds are the bottlenecks of system response time.

Furthermore, Wang Xianchao et al. constructed a service model for TOC by integrating it with M/M/1, M_X/M/1, and M/M_B/1 queuing systems²³. They proposed a computation completion scheduling strategy and its corresponding algorithm. The study discussed the computation methods for request reception time, preprocessing time, execution time, and transmission time based on various queuing systems. The total response time was obtained by summing these time components. Finally, numerical simulation models were utilized to evaluate response times under two scheduling strategies. The results demonstrate that the proposed computation completion scheduling strategy outperforms the immediate scheduling strategy.

These studies focus on performance optimization, the trade-off between performance and energy consumption remains underexplored^{24,25}.

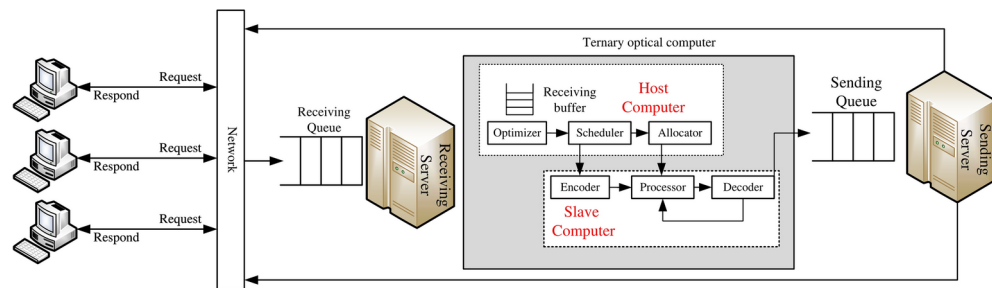


Fig. 2. Service paradigm of TOC.

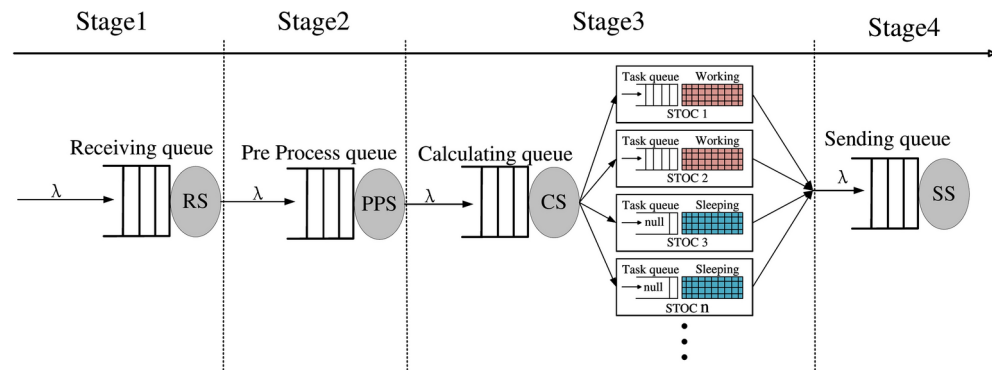


Fig. 3. Four-stage service model.

TOC service model

The TOC employs a service model similar to that of cloud computing²⁶, delivering services to users through a network. The paradigm of TOC service delivery is illustrated in Fig. 2. Like cloud computing, TOC enables users to submit computation-data files through terminals to obtain desired computational services, process the data, and return the results. However, unlike cloud computing, TOC is designed based on the Decrease-Radix Design Principle, granting it exceptional flexibility and scalability. Specifically, TOC primarily handles binary and ternary logic operations, either directly or indirectly²¹. Moreover, it features massive-bit parallelism¹⁸, enabling efficient large-scale computations²⁷. Additionally, TOC service requests exhibit significant diversity, and its processors can be dynamically reconfigured to adapt to varying computational demands^{28,29}. This architecture differs fundamentally from traditional electronic computers, which are often limited by fixed hardware structures that constrain efficient parallel processing. As a result, TOC provides users with a more flexible, efficient, and adaptable computing environment.

When a user submits a service request to the TOC, the user's tasks are initially encapsulated into a specialized format, referred to as the SZG file³⁰, where "SZG" originates from the Chinese pinyin SanZhiGuang, which means ternary optical. The system scheduler then manages these requests. The TOC's dynamic bit-wise reconfigurability and allocation enable exceptional parallel processing capabilities. Each optical processor can be further subdivided into multiple independent small optical processors, enabling the system to execute multiple tasks simultaneously in parallel³¹.

The Tandem Queuing model is particularly suitable for TOC because it mirrors the multi-stage, sequential nature of the computation and data processing processes in TOC. Specifically, the stages of task processing in TOC are closely linked, with each STOC in the system performing a distinct computation task before passing it on to the next. The Tandem Queuing system effectively captures this flow of tasks between stages, which allows for a more accurate representation of the service process in TOC.

By incorporating queueing theory, a four-stage tandem queueing system is developed. As depicted in Fig. 3, this service model consists of four sequential stages. The platform employed in this study is an extension of the SD16 architecture, offering a total capacity of 1152 data bits³². By using optical processor reconfigurable technology, the platform is evenly divided into n independent optical processors (OPs), resulting in the creation of n Small ternary optical computers (STOCs) available for user access.

In the first stage of the service process, users submit requests to the server using high-level language text. These requests may involve various computational tasks, such as data analysis, algorithmic processing, or other complex operations. The Receiving Service (RS) is the dedicated module responsible for handling these tasks, following the First Come, First Served (FCFS) rule. This approach ensures that tasks are queued and processed strictly in the order they arrive, promoting fairness and systematic management for all users. Upon receiving a request, the RS validates and organizes the tasks into an internal service queue, where they are securely stored

and prepared for further processing. At this stage, the TOC server accurately records and manages incoming user requests, ensuring they are appropriately prioritized and maintained.

In the second stage, user requests are sent to the data Pre-Processing Server (PPS) following the FCFS principle. During this stage, each task within the requests is converted into a ternary data format³⁰ to ensure compatibility with optical processing. This conversion is a critical step, as it aligns the tasks with the processing capabilities of the optical computers, thereby enhancing the efficiency of ternary data handling by the optical processors. Simultaneously, the TOC service system calculates the computational workload of each task, providing essential insights into its complexity and estimated processing time. This workload analysis is crucial for optimizing resource allocation and improving overall system processing efficiency. The main objectives of this stage are to accurately and promptly transform user requests into data suitable for optical processing and to prepare for subsequent task allocation and execution.

In the third stage, all tasks are sequentially dispatched to the Calculation Server (CS) based on their arrival order as recorded by the scheduler. Upon detecting an idle small optical processor within the system, the scheduler assigns a task to the available processor, allowing computation to commence promptly. This task allocation mechanism strictly adheres to the FCFS principle, ensuring fairness and maintaining the sequential order of task processing. Conversely, if the scheduler determines that no tasks remain pending for processing, the scheduling algorithm automatically ceases operation. This approach not only guarantees efficient utilization of computational resources but also minimizes redundant system activities, thereby enhancing overall processing efficiency.

In the fourth stage, the TOC service system transmits the processed data to users via the Sending Service Server (SS), adhering to the FCFS rule. This step concludes the service process, ensuring that all computational results are delivered in the order of task submission. This approach upholds fairness and completes the cycle of handling user requests.

Performance model

This study uses system response time (T) as the performance metric, where T is defined as the time required for the system to receive the computational results after a user submits a request. The formula for T is given by:

$$T = T_{RS} + T_{PPS} + T_{CS} + T_{SS} \quad (2)$$

T_i ($i = RS, PPS, CS, SS$) represents the time required for each stage of the model shown in Fig. 3. Specifically, T_{RS} denotes the queue reception time, T_{PPS} indicates the time required for data pre-processing, T_{CS} represents the total time spent on data computation and decoding, and T_{SS} refers to the data transmission time.

Task reception stage

The requests submitted by users are random and each request is independent of others, with no mutual influence³³. In this stage, the M/M/1 queuing model³⁴ can be applied for modeling. It is assumed that user requests arrive at a rate λ , meaning the inter-arrival times follow an exponential distribution with parameter λ . Additionally, it is assumed that the service rate of the receiving server RS follows an exponential distribution with parameter μ_{RS} . Based on the M/M/1 queuing model, the service of the RS in the first stage is modeled, and the corresponding Continuous-Time Markov Chain (CTMC) state transition diagram is shown below as Fig. 4.

At this point, the utilization of the task reception server can be expressed as:

$$\rho_{RS} = \frac{\lambda}{\mu_{RS}} \quad (3)$$

Assuming that the arrival probability of tasks and the service rate of the receiving server remain stable, both follow exponential distributions. When the average service rate exceeds the average arrival rate, it indicates that the system has sufficient time to process requests. In this case, the system will be in a steady state. In other words, when $\lambda/\mu_{RS} < 1$, the system will achieve stability, and this process conforms to the Markov queuing model. According to the state transition diagram of the M/M/1 queuing model, in the steady state, for any state n , the “inflow” equals the “outflow,” leading to the following equation:

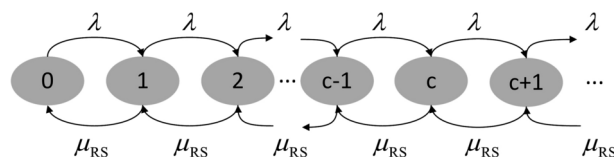


Fig. 4. State transition diagram of the M/M/1 queuing model.

$$\begin{cases} p_1 \mu_{RS} = p_0 \lambda \\ p_0 \lambda + p_2 \mu_{RS} = p_1 \mu_{RS} + p_1 \lambda \\ p_1 \lambda + p_3 \mu_{RS} = p_2 \mu_{RS} + p_2 \lambda \\ \vdots \\ p_{n-1} \lambda + p_{n+1} \mu_{RS} = p_n \mu_{RS} + p_n \lambda \end{cases} \quad (4)$$

From the first set of equations, it is evident that the balance conditions for each state are established through the transition rates between neighboring states. By observing these recursive relationships, we can progressively derive the probability distribution for each state. Next, by applying the recursive formula, we express the probabilities of each state in terms p_0 , yielding the following results:

$$\begin{cases} p_1 = \frac{\lambda}{\mu_{RS}} p_0 \\ p_2 = \frac{\lambda^2}{\mu_{RS}^2} p_0 \\ p_3 = \frac{\lambda^3}{\mu_{RS}^3} p_0 \\ \vdots \\ p_n = \frac{\lambda^n}{\mu_{RS}^n} p_0 \end{cases} \quad (5)$$

p_0 represents the probability of having zero tasks in the system, and p_2 represents the probability of having two tasks in the system. Let $p_m (m = 0, 1, 2, \dots)$ denote the probability of having m tasks in the system. The probability $p_m = \frac{\lambda^m}{\mu_{RS}^m} p_0$. Summing the probabilities of all possible states in the task reception system yields $p_0 + p_1 + p_2 + \dots + p_n = 1$, or $\sum_{i=0}^n p_i = 1$. Assuming the average number of requests in the system is N_{RS} :

$$N_{RS} = \sum_{m=0}^{\infty} m p_m = \rho_{RS} (1 - \rho_{RS}) \sum_{m=0}^{\infty} m \rho_{RS}^{m-1} = \frac{\lambda}{\mu_{RS} - \lambda} \quad (6)$$

According to Little's Law³⁵, the average time for receiving a task is:

$$T_{RS} = \frac{N_{RS}}{\lambda} = \frac{1}{\mu_{RS} - \lambda} \quad (7)$$

Data pre-processing stage

The time for a user request to enter the second stage follows an exponential distribution with parameter λ . During this stage, the primary task of the Pre-processing Service Server is to convert the operands into MSD format, with a service rate of μ_{PPS} . The modeling of PPS response time T_{PPS} can be referenced from the first stage. In this queuing process, the average queue length of tasks in the pre-processing queue is given by N_{PPS} , where $T_{PPS} = W + W_q$, with W being the time spent by the system servicing the task and W_q representing the waiting time of the task in the system. The total number of tasks in the system, N_{PPS} includes both the tasks currently being processed and those in the queue. The average response time $T_{PPS} = \frac{N_{PPS}}{\lambda}$.

$$\begin{cases} W = \frac{1}{\mu_{RS}} \\ W_q = \frac{\lambda}{\mu_{RS}(\mu_{RS} - \lambda)} \end{cases} \quad (8)$$

We begin by considering the service time W and the queuing time W_q , which represent the average time spent servicing a single request and the average time spent waiting in the queue, respectively. Using these parameters, we can further derive the total response time of the system, T_{PPS} . By definition, T_{PPS} is the sum of the service time and the queuing time. Applying this relationship, we arrive at the following expression for the total response time:

$$T_{PPS} = W + W_q = \frac{N_{PPS}}{\lambda} = \frac{1}{\mu_{PPS} - \lambda} \quad (9)$$

Data computation stage

The features of TOC lie in its numerous bits and bit-wise allocation, allowing different logic operators to be constructed on the computing units and enabling parallel processing of multiple requests. As the number of divisions increases, parallelism improves, but the service capacity of each STOC decreases. Conversely, when the

number of divisions decreases, parallelism reduces, but the service capacity of each STOC increases. Therefore, the choice of division strategy directly affects both the computation time and system energy consumption.

When the processors of TOC are divided into n STOCs, the modeling of the third stage can be referenced from the M/M/c queueing system, where n represents the division number of TOC and c denotes the number of active STOCs, with $c \leq n$. It is assumed that each STOC is identical in specifications with the same software and hardware configurations and that the probability of task assignment by the scheduler is equal for each STOC. The corresponding state transition diagram of the M/M/c queueing model is depicted in Fig. 5.

The service rate of the entire OP is μ , so the service rate of each STOC is $\mu_c = \frac{\mu}{n}$. When the arrival rate of computation requests is also λ , n represents the number of TOC divided equally into STOCs and c represents the number of activated STOCs. In this stage, the utilization rate of each STOC is $\rho_c = \frac{\lambda}{\mu_c}$. When $\rho_c < 1$, the system reaches a steady state. The probability that there are no tasks in the system, p_0 , can be expressed as:

$$p_0 = \left[\sum_{i=0}^{c-1} \frac{1}{i!} \left(\frac{\lambda}{\mu_c} \right)^i + \frac{1}{1 - \rho_c} \frac{1}{c!} \left(\frac{\lambda}{\mu_c} \right)^c \right]^{-1} \quad (10)$$

The probability p_m of having m tasks in the system for $m = 0, 1, 2, \dots$ is given by:

$$p_m = \begin{cases} \frac{1}{m!} \left(\frac{\lambda}{\mu_c} \right)^m p_0, & 0 < m \leq c \\ \frac{1}{c! c^{m-c}} \left(\frac{\lambda}{\mu_c} \right)^m p_0, & m > c \end{cases} \quad (11)$$

At this point, the queue length of the system, L_q , can be expressed as $L_q = \frac{1}{c!} \left(\frac{\lambda}{\mu_c} \right)^c \frac{\rho_c}{(1 - \rho_c)^2} p_0$. The total system length L_c , is $L_c = L_q + \frac{\lambda}{\mu_c}$. The total time T_c that a task spends in the computation stage is determined by the average number of tasks in the system, L_c :

$$L_c = \frac{1}{c!} \left(\frac{\lambda}{\mu_c} \right)^c \frac{\rho_c}{(1 - \rho_c)^2} p_0 + \frac{\lambda}{\mu_c} \quad (12)$$

The response time during the computation phase can be expressed as: $T_c = \frac{L_c}{\lambda}$.

Data transmission stage

In the final stage of service, the results computed by the upper-level system are aggregated and subsequently packaged by the data transmission server before being sent to the user terminal. During this period, the process can be modeled as a simple M/M/1 queueing system. The average service rate of the Sending Server (SS) is denoted by μ_{ss} . Based on the modeling of the task reception phase and pre-processing phase, the time expenditure for the data transmission phase is given by:

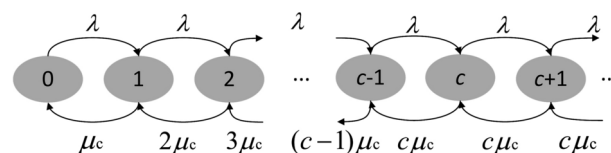
$$T_{ss} = \frac{1}{\mu_{ss} - \lambda} \quad (13)$$

By solving the equations for T_{RS} , T_{PPS} , T_{CS} , and T_{SS} , the overall response time T can be obtained.

Energy consumption model

In this study, the TOC service platform, an expansion of the SD16 architecture, comprises a total of 1152 data trits³². This platform can be partitioned into multiple smaller processors, thereby generating several STOC. While TOC processors are known for their low energy consumption, the increasing number of trits in these processors in recent years points to their potential evolution into cloud servers, data centers, or even supercomputers. However, this expansion presents significant challenges in managing large-scale data processing tasks, inevitably leading to higher energy consumption and substantial economic costs.

To explore the potential for reducing energy consumption while enhancing performance, this study proposes uniformly dividing the optical processors of the platform into n independent STOCs. Specifically, to achieve the dual objective of minimizing energy consumption and maintaining system performance, the TOC service platform integrates a sleep mode for inactive STOCs, thereby significantly reducing power usage. Meanwhile, particular attention is given to analyzing the energy consumption of active STOCs during this process. This approach ensures a balanced trade-off between performance optimization and energy efficiency, providing a feasible solution for sustainable computing.



Partitions of STOCs	Number of active STOCs									
	1	2	3	4	5	6	7	8	9	10
2	118.06	97.22								
3	187.50	117.44	113.01							
4	395.83	141.86	130.74	129.36						
5		173.61	149.62	146.40	145.91					
6		218.75	170.34	163.82	162.71	162.53				
7		291.26	193.92	181.87	179.66	179.25	179.18			
8		432.87	221.91	200.87	196.84	196.03	195.87	195.84		
9		851.97	256.84	221.26	214.40	212.90	212.58	212.51	212.50	
10			303.24	243.66	232.48	229.92	229.33	229.20	229.17	229.16

Table 3. System response time under low load with $\rho = 0.2$ (s).

Partitions of STOCs	Number of active STOCs									
	1	2	3	4	5	6	7	8	9	10
2	250.00	123.01								
3		161.46	137.25							
4		268.52	163.04	152.52						
5			203.70	173.91	168.33					
6			291.20	201.27	187.70	184.44				
7			711.39	241.67	210.05	202.75	200.75			
8				316.07	238.04	222.72	218.45	217.18		
9				528.74	277.30	245.62	237.14	134.51	233.69	
10					342.35	273.73	257.51	252.46	250.79	250.24

Table 4. System response time under medium-low load with $\rho = 0.4$ (s).

Let E denote the total energy consumption of the TOC system. Let E_{work} and E_{sleep} represent the power consumption of the STOCs during active and sleep modes, respectively. The total energy consumption E is then given by:

$$E = cE_{\text{work}} + (n - c)E_{\text{sleep}} \tag{14}$$

where c is the number of active STOCs, and n is the total number of STOCs. The power consumption of the backlight for each SD16 unit is approximately $150\text{ mA} \times 5\text{ V} = 750\text{ mW}$. Since the TOC service platform is expanded from 6 SD16 units, the total backlight power consumption is approximately 4.5 W.

Simulation experiment of the TOC service model
Impact of STOC partitioning and activation numbers on response time

This section aims to investigate the effects of different partitioning strategies and the number of activated STOCs on system response time under various load conditions. Special emphasis is placed on the role of the number of active STOCs. Based on the reconfigurability of TOC, the system can be partitioned into different numbers of STOCs, and the choice of partitioning strategy directly affects response time. By analyzing the relationship between these variables, this study seeks to elucidate how the quantity of STOCs influences system performance across different load scenarios, thereby providing a theoretical basis for optimizing response time.

To analyze the system response time under varying load conditions, task arrival rates were set according to 20%, 40%, 60%, 80%, and 90% of the system’s service capacity, representing low, medium-low, medium, medium-high, and high load conditions, respectively. The measured response times for each load condition are summarized in Tables 1, 2, 3, 4, 5, with Table 3 corresponding to the low load condition, Table 4 to the medium-low load condition, Table 5 to the medium load condition, Table 6 to the medium-high load condition, and Table 7 to the high load condition.

In queuing systems, the system load factor (ρ) is defined as the ratio of the arrival rate (λ) to the service rate (μ), represented as:

$$\rho = \frac{\lambda}{\mu}$$

In the third stage, the switching of STOCs can be controlled to enhance system service capacity, enabling it to meet demands under varying load conditions. Consequently,

Partitions of STOCs	Number of active STOCs									
	1	2	3	4	5	6	7	8	9	10
2		177.08								
3		388.16	189.78							
4			263.58	203.63						
5				250.79	218.17					
6				421.94	254.31	233.19				
7					334.09	263.44	248.54			
8					859.47	315.86	275.20	264.14		
9						460.03	315.12	288.43	279.94	
10							393.97	321.42	302.55	295.89

Table 5. System response time under medium load with $\rho = 0.6$ (s).

Partitions of STOCs	Number of active STOCs									
	1	2	3	4	5	6	7	8	9	10
2		342.59								
3			353.93							
4				366.36						
5					379.51					
6					800.85	393.15				
7						606.64	407.16			
8							551.61	421.47		
9								531.54	436.02	
10									525.55	450.77

Table 6. System response time under medium-high load with $\rho = 0.8$ (s).

Partitions of STOCs	Number of active STOCs									
	1	2	3	4	5	6	7	8	9	10
2		675.44								
3			686.18							
4				697.96						
5					710.42					
6						723.35				
7							736.66			
8								750.26		
9									764.10	
10										778.12

Table 7. System response time under high load with $\rho = 0.9$ (s).

$$\rho = \frac{\lambda}{\mu} < \frac{\lambda}{c\mu_c} < 1 \quad (15)$$

where c denotes the number of servers. For a stable system, ρ should remain less than 1 (i.e., $\rho < 1$), which implies that the average arrival rate will not exceed the service capacity. This ensures that, over the extended operation, the system will not accumulate an infinite queue length, thereby maintaining stability and avoiding potential collapse. When the “Number of Active STOCs” in the table is less than the “Partitions of STOCs,” certain data entries are left blank because, in these cases, the arrival rate exceeds the service capacity (i.e., $c\mu_c < \lambda$). This indicates that the arrival rate (demand) surpasses the system’s capacity to serve. In such instances, the system, on average, cannot fully process all incoming requests per time unit, resulting in persistent occupation of resources and leaving new requests with minimal chance of timely processing. Consequently, the system trends toward saturation or even collapse. Therefore, scenarios where the arrival rate exceeds service capacity are excluded from consideration.

From Fig. 6, it is evident that under the same partitioning conditions, as the number of STOCs increases, the system’s response time gradually decreases, although the rate of reduction becomes increasingly gradual. In an

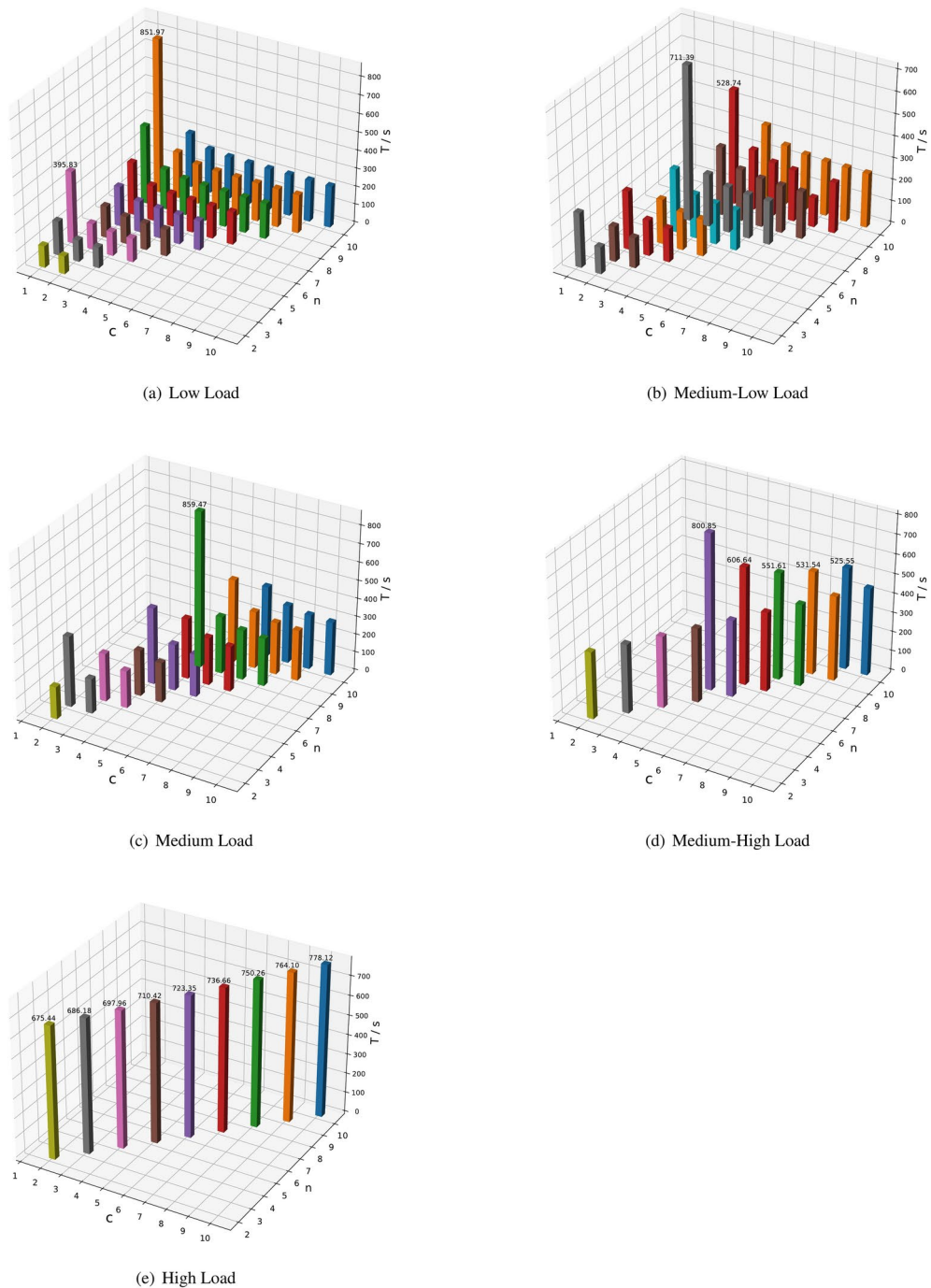


Fig. 6. Response time under different load conditions.

M/M/c queuing model, an increase in the number of servers typically leads to a reduction in system response time. More servers allow for the parallel processing of a greater number of tasks, thereby reducing the waiting time of tasks in the queue. Hence, increasing c generally enhances the overall system efficiency, reflected by a decrease in response time. However, this improvement is not linear. Once c reaches a certain critical point, the performance enhancement becomes less pronounced as the waiting time approaches its minimum value.

Additionally, in Fig. 6a–c, there are instances where the response time is significantly higher than others. This occurs when the system service rate is slightly higher than the task arrival rate. In an M/M/c queuing model, when the task arrival rate approaches the service rate, the system is nearly saturated. Under such conditions, the server utilization is very high, with nearly all servers continuously handling tasks. This leads to a backlog of tasks in the queue, causing the queue length to increase rapidly and requiring new tasks to wait longer for processing. Consequently, the average waiting time of the system is markedly extended. Therefore, when the arrival rate is

μ_c	c	W_q	$\frac{1}{\mu_c}$	T
120	1	60.00	30.00	90.00
60	2	48.00	60.00	108.00
40	3	40.00	90.00	130.00
30	4	34.06	120.00	154.06
24	5	29.40	150.00	179.40
20	3	25.63	180.00	205.63

Table 8. Verification experiment.

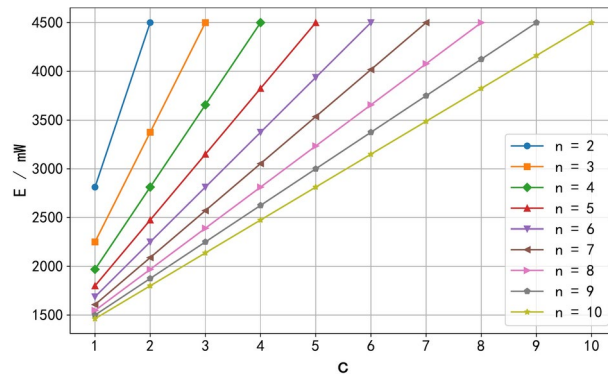


Fig. 7. Relationship between energy consumption E , number of partitions n , and active processors c .

close to the service rate, the system's performance deteriorates significantly, resulting in a considerable increase in response time.

Furthermore, an interesting phenomenon is observed: with a fixed task arrival rate and total system service rate, increasing the number of partitions leads to a longer system response time. This phenomenon is intuitively illustrated in Fig. 6d,e. To investigate this phenomenon, a verification experiment was conducted using an M/M/c queueing system, as summarized in Table 8. The parameters were set as follows: task arrival rate is $\lambda = 80$ tasks per hour, and the service rate is $\mu = 120$ tasks per hour. The parameter c represents the number of active STOCs, with the condition that $c \cdot \mu_c = \mu$, where μ_c denotes the service rate of a single STOC. W_q represents the average queueing time of a task, while $\frac{1}{\mu_c}$ is the service time per task handled by an individual STOC. T denotes the system's response time.

From the table, it is clearly observed that the system response time T , the average queueing time W_q , and the task service time $\frac{1}{\mu_c}$ all vary with the number of partitioning STOCs (which is also the number of all active STOCs c). Specifically, when all STOCs are activated, as the number of partitioning STOCs increases, the average queueing time W_q significantly decreases from 60 seconds to approximately 25 seconds. This indicates that increasing the number of service stations helps to effectively reduce queueing delays and improve the overall system service capacity. However, increasing the number of STOCs also brings a noticeable side effect: the performance of each individual STOC declines, leading to an increase in the time required to process each task, $\frac{1}{\mu_c}$.

Furthermore, the system response time T is the sum of the average queueing time W_q and the task service time $\frac{1}{\mu_c}$. It can be intuitively seen from the table that the system response time T is largely influenced by the task service time $\frac{1}{\mu_c}$. Although increasing the number of evenly distributed STOCs significantly reduces the average queueing time, the increase in task service time greatly offsets this advantage, resulting in an increase in the system response time.

Therefore, the analysis indicates that a higher number of evenly partitioning STOCs is not always better. In practical applications, it is necessary to find a balance between the number of service stations and the performance of each individual server to achieve optimal system response time.

Impact of STOC partitioning and activation on system energy consumption

Based on the relationship between system energy consumption, the number of partitions, and the number of active processors, $E = cE_{\text{work}} + (n - c)E_{\text{sleep}}$, an energy consumption graph is plotted as Fig. 7.

When the number of service stations c increases, the opportunity for each task to wait for service decreases, leading to a reduction in the average waiting queue length L_q . This is because more service stations mean that more tasks can be handled concurrently, reducing the time tasks spend waiting in the queue. However, as the number of service stations increases, the total number of busy service stations $L_{\text{busy}} = \rho_1 = \frac{\lambda}{\mu_c}$ also rises. The

total number of tasks in the system L_s , which equals the number of tasks in the queue L_q plus the number of tasks being served L_{busy} increases because although L_q decreases with the increase in c , the increase in L_{busy} is more significant. Consequently, the total number of tasks L_s in the system increases with the number of service stations c .

It is evident from the Fig. 7 that the greater the number of TOC partitions, the more precise the control over power consumption. When the system load is approximately 10%, a server divided into two STOCs must keep one STOC active, resulting in noticeably higher power consumption compared to a server divided into 10 STOCs, where only one STOC is active and nine are in sleep mode. Similarly, when the system load is around 60%, a server divided into two STOCs needs both STOCs to remain active, whereas a server divided into six STOCs only requires five to be active, with one in sleep mode. In this case, the power consumption is lower than when the system is divided into two STOCs. Thus, the greater the number of partitions, the more beneficial it is for fine-tuned control over energy consumption.

Bi-objective optimization of performance and energy consumption

In studying the relationship between the performance and energy consumption of the TOC service system, it was observed that as the number of active STOCs (denoted as c) increases, the system's response time decreases, indicating an enhancement in performance. This suggests that with the increase in the number of processors, task execution times are relatively shortened, leading to an overall improvement in system performance. However, this performance gain is accompanied by an increase in total energy consumption, as the activation of more processors results in higher energy usage, thereby driving up the overall energy cost.

Evidently, there exists a trade-off between system performance and energy consumption. Increasing the number of processors can enhance parallel processing capabilities and improve execution efficiency, thereby reducing task completion time. However, this also results in greater energy consumption. Thus, when designing and optimizing the system, it is crucial to account for this performance-energy trade-off and determine the optimal number of processors to meet specific performance goals while adhering to energy constraints.

Both response time and energy consumption should be regarded as distinct objective functions, where improving one often negatively impacts the other. Consequently, a balance between these competing objectives must be achieved. For instance, increasing the number of processors to reduce overall execution time may lead to higher total energy consumption, whereas lowering energy usage might prolong task execution time. Therefore, identifying the optimal number of processors necessitates a comprehensive consideration of these factors to achieve an optimal balance between performance and energy consumption.

For simplicity, the optimization of the TOC service model focuses on the M/M/c model for the third stage. By controlling the number of STOCs and their operational status, the goal is to achieve multi-objective optimization of both response time and energy consumption.

Let h represent the energy consumption per unit time for each STOC, and ω denote the energy consumed per unit time that each task spends in the system. The objective function can be defined as:

$$f(c_n) = hc_n + \omega L_s \quad (16)$$

where c_n represents the state of the system with the optical processor divided into n parts and c STOCs. Both c and n are discrete, and it must hold that $\lambda/c\mu_n < 1$. To determine the optimal c_n , marginal analysis can be employed, where the optimal C_n^* must satisfy:

$$\begin{cases} f(c_n^*) < f(c_n^* - 1) \\ f(c_n^*) < f(c_n^* + 1) \end{cases} \quad (17)$$

Substituting Eq. (16) into Eq. (17), we obtain:

$$\begin{cases} hc_n^* + \omega L_s(c_n^*) < h(c_n^* - 1) + \omega L_s(c_n^* - 1) \\ hc_n^* + \omega L_s(c_n^*) < h(c_n^* + 1) + \omega L_s(c_n^* + 1) \end{cases} \quad (18)$$

Simplifying, we get:

$$L_s(c_n^*) - L_s(c_n^* + 1) < \frac{h}{\omega} < L_s(c_n^* - 1) - L_s(c_n^*) \quad (19)$$

For the third stage M/M/c model, calculations will be performed to find the optimal c_n^* , such that the response time is minimized while maintaining the system energy consumption at a lower level. The queue length under low load conditions and with a partition number of 10, calculated using Eqs. (16) and (19), is presented in Table 9.

Under low load, when the partition quantity is 10, each additional small optical processor incurs an extra power consumption of 350 mW. The energy cost per unit time for each task remaining in the system, denoted as ω , is approximately 500 mW. With $\frac{h}{\omega} = 0.7$, which falls within the interval (0.1341, 0.7150), the optimal number of active STOCs $c_n^* = 4$.

Similarly, the optimal value of c_n^* for other partition scenarios under different loads can be calculated accordingly, as shown in Tables 10, 11, 12, 13 and 14, corresponding to low, medium-low, medium, medium-high, and high load conditions, respectively.

From Table 10, it can be observed that under a low load the system does not require extensive parallel processing capabilities. The impact of the partition number and the number of activated STOCs on the response

c_n	$L_s(c_n)$	$L_s(c_n) - L_s(c_n + 1) - L_s(c_n - 1) - L_s(c_n)$
1	–	–
2	–	–
3	2.8889	0.7150– $+\infty$
4	2.1739	0.1341–0.7150
5	2.0398	0.0308–0.1341
6	2.0090	0.0071–0.0308
7	2.0019	0.0015–0.0071
8	2.0004	0.0003–0.0015
9	2.0001	0.0001–0.0003
10	2.0000	0–0.0001

Table 9. Queue lengths with different numbers of active STOCs when the partition quantity is 10 under low load. Significant values are in bold.

n	c_n^*	T	E	$f(c_n^*)$
2	1	118.06	2812.50	2083.33
3	1	187.50	2250.00	1916.67
4	2	141.87	2812.50	2226.19
5	2	173.61	2475.00	2066.67
6	2	218.75	2250.00	2104.17
7	3	193.92	2571.43	2288.53
8	3	221.91	2390.63	2268.96
9	3	256.84	2250.00	2332.72
10	4	243.66	2475.00	2486.96

Table 10. Low Load($\lambda = \mu * 20\%$). Significant values are in bold.

n	c_n^*	T	E	$f(c_n^*)$
2	1	250.00	2812.50	3750.00
3	2	161.46	3375.00	3270.83
4	3	163.04	3656.25	3581.46
5	3	203.70	3150.00	3544.45
6	4	201.27	3375.00	3748.62
7	4	241.67	3053.57	3900.10
8	5	238.04	3234.38	4043.99
9	5	277.30	3000.00	4272.09
10	6	273.73	3150.00	4384.76

Table 11. Medium-low Load($\lambda = \mu * 40\%$). Significant values are in bold.

time is much more significant than on energy consumption; in fact, activating too many STOCs leads to unnecessary power usage. Dividing the TOC into 3 parts and activating only 1 STOC can effectively reduce energy consumption while maintaining a moderate response time.

When the load increases to a medium-low level, as shown in Table 11, the system's demand for processing capacity rises slightly. Under these conditions, activating only 1 STOC is no longer sufficient to meet the required response speed, so the number of activated STOCs needs to be increased. The data indicate that partitioning the system into 3 STOCs and activating 2 of them can better handle the increased load, resulting in a significant improvement in response time. Although the energy consumption increases somewhat, the overall performance remains optimal, demonstrating that activating additional STOCs under these conditions can better balance performance and energy consumption.

As the load further increases to medium levels, as evidenced by Tables 12 and 13, the system requires a higher degree of parallel processing capability. Reducing the partition number by dividing the system into 2 STOCs and activating both fully utilizes the available resources, thereby better controlling both response time and energy consumption.

In high-load scenarios, as shown in Table 14, the system's demand for processing capacity surges dramatically. Regardless of the partition number, all STOCs must be activated to maintain stable system operation, resulting

n	c_n^*	T	E	$f(c_n^*)$
2	2	177.08	4500.00	4437.50
3	3	189.78	4500.00	4666.60
4	4	203.63	4500.00	4915.28
5	4	250.79	3825.00	5064.15
6	5	254.31	3937.50	5244.32
7	6	263.44	4017.85	5491.97
8	7	275.20	4078.13	5766.15
9	8	288.43	4125.00	6052.78
10	8	321.42	3825.00	6335.47

Table 12. Medium Load($\lambda = \mu * 60\%$). Significant values are in bold.

n	c_n^*	T	E	$f(c_n^*)$
2	2	342.59	4500.00	5722.22
3	3	353.93	4500.00	5994.38
4	4	366.37	4500.00	6292.87
5	5	379.51	4500.00	6608.23
6	6	393.15	4500.00	6935.54
7	7	407.16	4500.00	7271.88
8	8	421.47	4500.00	7615.29
9	9	436.02	4500.00	7964.45
10	10	450.77	4500.00	8318.36

Table 13. Medium-high Load($\lambda = \mu * 80\%$). Significant values are in bold.

n	c_n^*	T	E	$f(c_n^*)$
2	2	675.44	4500.00	8236.84
3	3	686.18	4500.00	8526.78
4	4	697.96	4500.00	8844.89
5	5	710.42	4500.00	9181.22
6	6	723.35	4500.00	9530.57
7	7	736.66	4500.00	9889.81
8	8	750.26	4500.00	10256.90
9	9	764.09	4500.00	10630.41
10	10	778.12	4500.00	11009.30

Table 14. High Load($\lambda = \mu * 90\%$). Significant values are in bold.

in maximum energy consumption when all STOCs are active. Moreover, increasing the partition number leads to a rapid rise in response time. Therefore, under high-load conditions, the system cannot further reduce the response time and energy consumption simply by increasing the partition number.

Overall, as the load increases, the optimal configuration shifts from “more partitions with fewer activations” to “fewer partitions with full activation.” In addition, merely increasing the number of partitions does not necessarily improve the response time; it may instead introduce additional coordination overhead. The data from these five tables clearly demonstrate how to adjust the strategy of combining the STOC partition number n and the number of activated STOCs c_n under different load conditions to achieve the best balance between response time and energy consumption.

In previous studies, researchers mainly focused on enhancing the parallel processing capability of optical processors through reconfiguration and partitioning techniques, thereby improving system performance. In contrast, our study not only focuses on performance but also places significant emphasis on the energy consumption issues that arise with performance improvements in today’s era of ever-increasing data volumes. This trade-off between performance and energy consumption enables the system to boost computational capacity while effectively controlling energy usage, in line with the demands for green computing in the big data era.

In our research, we considered the impact of both the partition number and the number of activated STOCs on the system’s energy consumption. By jointly tuning these two parameters, we can achieve a better balance between response time and energy consumption, potentially reducing the system’s energy usage significantly

without sacrificing performance. Consequently, our optimization of performance and energy consumption in TOC exhibits superior results compared to previous studies.

Summary

As TOC technology advances toward large-scale practical applications, balancing performance optimization with energy consumption control becomes a critical step in realizing its widespread adoption. Based on M/M/1 and M/M/n models, this study conducts an in-depth analysis of the service response times at various stages of ternary optical computing and the energy consumption characteristics of different system components. By establishing an optimization model between execution time and energy consumption, the study aims to reduce energy usage while ensuring high system performance, thus optimizing system configurations to achieve a dual-objective optimization of both performance and energy efficiency. Specifically, through rational adjustment of system resource allocation, the analysis explores how to strike the optimal balance between performance and energy efficiency under different load conditions.

The proposed research mechanism aims to optimize both performance and energy consumption in TOC systems, but several limitations should be noted. First, the underlying model relies on simplified assumptions regarding arrival processes, service time distributions, and queueing behavior. Such assumptions, while beneficial for analytical tractability, may not fully capture the complexities and variability encountered in real-world scenarios. Moreover, the optimization results depend heavily on the specific configuration of the STOC partition number and the number of activated STOCs. This sensitivity means that the optimal settings derived under controlled conditions might not hold in environments with dynamic or unpredictable workloads. In our future research, we will work to address this limitation.

Throughout the research, this study not only conducts detailed analysis and calculations for various model parameters but also validates the effectiveness of the optimization model through experiments. The resulting strategy significantly reduces system energy consumption under certain load conditions, while maintaining or even enhancing overall system performance. Furthermore, we discovered that the ARM9 control unit in the SD16 supports Dynamic Voltage and Frequency Scaling (DVFS)³⁶. This technology allows real-time adjustment of the chip's operating frequency and voltage to adapt to varying workload conditions, effectively reducing the chip's energy consumption while delivering the required performance. Future research will continue to explore the performance and energy consumption of TOC based on more complex models such as tandem queueing models and DVFS.

Data availability

The authors confirm that all data supporting the findings and research data of this study are available in the article.

Received: 17 December 2024; Accepted: 25 April 2025

Published online: 29 April 2025

References

- Yi, J. Draw near optical computer. *J. Shanghai Univ.* (2011).
- Zhang, S., Peng, J., Shen, Y. & Wang, X. Programming model and implementation mechanism for ternary optical computer. *Opt. Commun.* **428**, 26–34. <https://doi.org/10.1016/j.optcom.2018.07.038> (2018).
- JunYong, Y. *Decrease-Radix Design Principle*. Ph.D. thesis (School of Computer Engineering and Science Shanghai University, 2009).
- Wang, Z. et al. Implementation and improvement of sd11 decoder. *J. Shanghai Univ. (Nat. Sci. Ed.)* **21**, 245. <https://doi.org/10.3969/j.issn.1007-2861.2013.07.053> (2015).
- Yi, J. et al. Theory and structure of ternary optical logic unit of sd16. *Acta Electron. Sin.* **51**, 1154–1162. <https://doi.org/10.12263/DZXB.20221096> (2023).
- Zhang, S., Shen, Y. & Zhao, Z. Design and implementation of a three-lane ca traffic flow model on ternary optical computer. *Opt. Commun.* **470**, 125750. <https://doi.org/10.1016/j.optcom.2020.125750> (2020).
- Song, K., Li, W., Zhang, B., Yan, L. & Wang, X. Parallel design and implementation of jacobi iterative algorithm based on ternary optical computer. *J. Supercomput.* **78**, 14965–14990. <https://doi.org/10.1007/s11227-022-04471-x> (2022).
- Wang, Z., Shen, Y., Li, S. & Wang, S. A fine-grained fast parallel genetic algorithm based on a ternary optical computer for solving traveling salesman problem. *J. Supercomput.* **79**, 4760–4790. <https://doi.org/10.1007/s11227-022-04813-9> (2023).
- Fang, L. et al. Design and implementation of pso based on ternary optical computer. In *Tenth International Conference on Applications and Techniques in Cyber Intelligence (ICATCI 2022)* (eds Abawajy, J. H., Xu, Z., Atiquzzaman, M. & Zhang, X.), 453–461 (Springer International Publishing, 2023).
- Erlang, A. K. The theory of probabilities and telephone conversations. *Nyt. Tidsskr. Mat. Ser. B* **20**, 33–39 (1909).
- Ziolkowski, M. On applications of computer algebra systems in queueing theory calculations. *Bull. Pol. Acad. Sci.-Tech. Sci.* **72**. <https://doi.org/10.24425/bpasts.2024.150199> (2024).
- Lin, J.-S. & Huang, C.-Y. Queueing-based simulation for software reliability analysis. *IEEE Access* **10**, 107729–107747. <https://doi.org/10.1109/ACCESS.2022.3213271> (2022).
- Alotaibi, F. M., Ullah, I. & Ahmad, S. Modeling and performance evaluation of multi-class queueing system with qos and priority constraints. *Electronics* **10**. <https://doi.org/10.3390/electronics10040500> (2021).
- Ye, C., Kong, S., Fu, Y. & Peng, J. Optical computer based application platform for msd multiplication. *Opt. Commun.* **458**, 124814. <https://doi.org/10.1016/j.optcom.2019.124814> (2020).
- Song, K., Ma, H., Zhang, H. & Yan, L. Research of relu output device in ternary optical computer based on parallel fully connected layer. *J. Supercomput.* **80**, 7269–7292. <https://doi.org/10.1007/s11227-023-05737-8> (2024).
- Zhehe, W. & Yunfu, S. Design and implementation of the walsh-hadamard transform on a ternary optical computer. *Appl. Opt.* **60**, 9254–9262. <https://doi.org/10.1364/AO.435457> (2021).
- Wang, X. et al. Performance analysis and evaluation of ternary optical computer based on asynchronous multiple vacations. *Soft Comput.* **27**, 4107–4123. <https://doi.org/10.1007/s00500-021-06656-7> (2023).
- Yi, J. et al. Ternary optical computer. *Chin. J. Nat.* **41**, 207–218 (2019).

19. Zhang, S. *et al.* Design and implementation of the dual-center programming platform for ternary optical computer and electronic computer. *Sci. Rep.* **14**. <https://doi.org/10.1038/s41598-024-75976-z> (2024).
20. Avizienis, A. Signed-digit number representations for fast parallel arithmetic. *IRE Trans. Electron. Comput.* **EC-10**, 389–400. <https://doi.org/10.1109/TEC.1961.5219227> (1961).
21. Jin, Y. *et al.* Principles and construction of msd adder in ternary optical computer. *Sci. China Inf. Sci.* **53**, 2159–2168. <https://doi.org/10.1007/s11432-010-4091-9> (2010).
22. Wang, X., Zhang, S., Zhang, M., Zhao, J. & Niu, X. Performance analysis of a ternary optical computer based on m/m/1 queueing system. In *Algorithms and Architectures for Parallel Processing* (eds Ibrahim, S., Choo, K.-K. R., Yan, Z. & Pedrycz, W.), 331–344 (Springer International Publishing, 2017).
23. Wang, X., Zhang, J., Gao, S., Zhang, M. & Wang, X. Performance analysis and evaluation of ternary optical computer based on a queueing system with synchronous multi-vacations. *IEEE Access* **8**, 67214–67227. <https://doi.org/10.1109/ACCESS.2020.2983773> (2020).
24. Wang, X. *et al.* Performance analysis and evaluation of ternary optical computer based on asynchronous multiple vacations. *Soft Comput.* **27**, 4107–4123. <https://doi.org/10.1007/s00500-021-06656-7> (2023).
25. Zhang, J., Liu, J., Wang, X., Song, K. & Wang, X. Ternary optical computer task dispatch model based on priority queueing system. In *2020 International Conference on Applications and Techniques in Cyber Intelligence* (eds Abawajy, J. H., Choo, K.-K. R., Xu, Z. & Atiquzzaman, M.), 674–682 (Springer International Publishing, Cham, 2021).
26. Qun, X. & Xianchao, W. Service model and performance analysis of ternary optical computer based on complex queueing system. *J. Natl. Univ. Defense Technol.* **39**, 140. <https://doi.org/10.11887/j.cn.201702021> (2017).
27. Kai, S. & LiPing, Y. Reconfigurable ternary optical processor based on row operation unit. *Opt. Commun.* **350**, 6–12. <https://doi.org/10.1016/j.optcom.2015.03.080> (2015).
28. Yan, J., Jin, Y. & Zuo, K. Decrease-radix design principle for carrying/borrowing free multi-valued and application in ternary optical computer. *Sci. China Ser. F Inf. Sci.* **51**, 1415–1426. <https://doi.org/10.1007/s11432-008-0140-z> (2008).
29. Shen, Z., Wu, L. & Yan, J. The reconfigurable module of ternary optical computer. *Optik* **124**, 1415–1419. <https://doi.org/10.1016/j.jileo.2012.03.081> (2013).
30. Li, S. & Jin, Y. Simple structured data initial szg file's generation software design and implementation. In *Proceedings of the 3rd International Conference on Wireless Communication and Sensor Networks (WCSN 2016)*, 383–388. <https://doi.org/10.2991/icwscn-16.2017.82> (Atlantis Press, 2016).
31. Jin, Y. *et al.* Principles, structures, and implementation of reconfigurable ternary optical processors. *Sci. China Inf. Sci.* **54**, 2236–2246. <https://doi.org/10.1007/s11432-011-4446-x> (2011).
32. Yi, J. *et al.* Theory and structure of ternary optical logic unit of sd16. *Acta Electron. Sin.* **51**, 1154–1162. <https://doi.org/10.12263/DZXB.20221096> (2023).
33. Ross, S. M. 4—Markov chains. In *Introduction to Probability Models*, 13th edn (ed Ross, S. M.), 201–299. <https://doi.org/10.1016/B978-0-44-318761-2.00009-9> (Academic Press, 2024).
34. Hirbod, F., Eshghali, M., Sheikhasadi, M., Jolai, F. & Aghsami, A. A state-dependent M/M/1 queueing location-allocation model for vaccine distribution using metaheuristic algorithms. *J. Comput. Des. Eng.* **10**, 1507–1530. <https://doi.org/10.1093/jcde/qwad058> (2023).
35. Little, J. D. C. A proof for the queueing formula: $L = \lambda W$. *Oper. Res.* **9**, 383–387. <https://doi.org/10.1287/opre.9.3.383> (1961).
36. Ramegowda, D. & Lin, M. Can learning-based hybrid dvfs technique adapt to different linux embedded platforms? In *2021 IEEE SmartWorld, Ubiquitous Intelligence and Computing, Advanced and Trusted Computing, Scalable Computing & Communications, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/IOP/SCI)*, 170–177. <https://doi.org/10.1109/SWC50871.2021.00032> (2021).

Acknowledgements

We sincerely thank Prof. WANG Xianchao, LI Shuang, ZHANG Jie, and all members of the TOC team for their constructive support and insightful discussions during the preparation of this paper. This research was partially supported by the National Natural Science Foundation of China (Grant Nos. 61672006 and 62302307); the Major Project of Natural Science Research in Anhui (Grant No. 2023AH040062); the Quality Engineering Projects in Anhui (Grant Nos. 2023cxtd068, 2023yjsmsgzs032, S202310371092); the Quality Engineering Project of Fuyang Normal University (Grant No. 2022JXTD0003); and the Natural Science Research Key Project of Fuyang Normal University (Grant No. 2022FSKJ04ZD).

Author contributions

Zhang Heqiang drafted the original manuscript, while Zhang Jie designed and constructed the four-stage tandem queueing system, and Liu Meng designed the experiments. Shi Wenqiang and Liu Weiwen developed the code and performed the experiments. Li Shuang refined the experimental data and analyzed the results. Professor Wang Xianchao provided guidance on the manuscript and experimental process and offered both equipment support and financial assistance for the project.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to X.W.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025